

Algorytmy genetyczne i sztuczne sieci neuronowe

Github link:

https://github.com/Baz00k/DSW_genetic_algorithms_and_neural_networks

Ćwiczenie 2: Perceptron

Wykonali Jakub Buzuk 49445 i Maksym Belilovskyi 49438

Perceptron

load_data.py

Importy i zmienne globalne:

```
import pandas as pd

IRIS_DATASET_URL = (
    "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
)
```

- Import pandas: Biblioteka pandas służy do przetwarzania i manipulacji danymi tabelarycznymi.
- Stała IRIS_DATASET_URL: Zawiera URL do zbioru danych Iris, dostępnego w UCI Machine Learning Repository.

Funkcja load_iris_data:

```
def load_iris_data(url=IRIS_DATASET_URL):
    """Load iris dataset from URL."""
    columns = [
        "Długość działki",
        "Szerokość działki",
        "Długość płatka",
        "Szerokość płatka",
        "Etykieta",
    ]
    return pd.read_csv(url, header=None, names=columns)
```

- Argument url:
- Domyślnie wskazuje na adres URL zbioru danych Iris.

- Można podać inny URL, jeśli chcemy wczytać dane z innego źródła.
- Definicja kolumn:
- Zmienna `columns` definiuje nazwy kolumn odpowiadające cechom i etykiatom kwiatów w zbiorze danych:
 - Długość działki, Szerokość działki: cechy dotyczące działki kwiatowej.
 - Długość płatka, Szerokość płatka: cechy dotyczące płatka kwiatowego.
 - Etykieta: określa gatunek kwiatu (`Iris-setosa`, `Iris-versicolor`, `Iris-virginica`).
- Funkcja wykorzystuje `pd.read_csv` do załadowania danych z pliku CSV.
- Argument `header=None` informuje, że dane nie mają nagłówka.
- Argument `names=columns` przypisuje wcześniej zdefiniowane nazwy kolumn.
- Funkcja zwraca obiekt `DataFrame` zawierający dane ze zbioru `Iris`

Funkcja `prepare_binary_classification_data`

```
def prepare_binary_classification_data(data, features=None):
    """Prepare data for binary classification."""

    label_setosa = "Iris-setosa"
    label_virginica = "Iris-virginica"

    binary_data = data[data["Etykieta"].isin([label_setosa,
label_virginica])].copy()

    # Map classes to -1 and 1
    binary_data["Etykieta"] = binary_data["Etykieta"].map(
        {label_setosa: -1, label_virginica: 1}
    )

    if features is None:
        features = ["Długość działki", "Długość płatka"]

    x = binary_data[features].values
    y = binary_data["Etykieta"].values

    return x, y
```

- `data`: `DataFrame` wczytany przez `load_iris_data`.
- `features`: lista kolumn (cech), które mają być użyte w klasyfikacji. Jeśli nie zostanie podana, domyślnie wybrane są Długość działki i Długość płatka.

- Funkcja ogranicza dane do dwóch gatunków: *Iris-setosa* i *Iris-virginica*, za pomocą metody `isin`.
- Etykieta *Iris-setosa* jest mapowana na -1.
- Etykieta *Iris-virginica* jest mapowana na 1.
- Jeśli lista `features` nie jest podana, wybierane są domyślnie dwie cechy: Długość działki i Długość płatka.
- `x`: tablica wartości cech (numpy array), zawierająca wybrane kolumny.
- `y`: tablica wartości etykiet (numpy array).
- Funkcja zwraca przetworzone dane wejściowe (`x`) i odpowiedzi (`y`), gotowe do wykorzystania w klasyfikacji.

Funkcja `get_iris_binary_data`

```
def get_iris_binary_data(features=None):
    """Convenience function to get preprocessed iris data for binary
    classification."""
    data = load_iris_data()
    return prepare_binary_classification_data(data, features=features)
```

- Funkcja najpierw wywołuje `load_iris_data`, aby wczytać dane z domyślnego URL (lub innego, jeśli zostanie podany w `IRIS_DATASET_URL`).
- Następnie wywołuje `prepare_binary_classification_data`, aby przetworzyć dane:
 - Filtruje odpowiednie gatunki (*Iris-setosa* i *Iris-virginica*).
 - Mapuje etykiety na wartości binarne.
 - Wybiera kolumny odpowiadające cechom.
- Funkcja zwraca gotowe dane (`x`, `y`), które można bezpośrednio wykorzystać w klasyfikacji.

Perceptron.py

Importy i zależności

```
import numpy as np
import matplotlib.pyplot as plt

from load_data import get_iris_binary_data
```

`numpy (np)`: Biblioteka do obliczeń numerycznych, używana do manipulacji macierzami i wektorami.

`matplotlib.pyplot (plt)`: Narzędzie do tworzenia wizualizacji danych, takich jak wykresy.

get_iris_binary_data: Funkcja z pliku load_data.py, która wczytuje i przygotowuje dane zbioru Iris do klasyfikacji binarnej.

Definicja klasy Perceptron

```
class Perceptron:
    """Klasyfikator - perceptron."""

    def __init__(self, eta=0.01, n_iter=10):
        self.eta = eta
        self.n_iter = n_iter
        self.w_ = None
        self.errors_ = []
```

eta: Współczynnik uczenia określa, jak duże zmiany są wprowadzane w wagach w trakcie aktualizacji.

n_iter: Liczba epok, czyli pełnych przejść przez dane uczące.

w_: Wektor wag, inicjalizowany później w metodzie fit.

errors_: Lista przechowująca liczbę błędnych klasyfikacji w każdej epoce, co pozwala na ocenę postępu uczenia.

Metoda fit

```
def fit(self, x, y):
    """Dopasowanie danych uczących."""
    self.w_ = np.zeros(1 + x.shape[1]) # Wektor wag z dodatkowym
    elementem na bias
    self.errors_ = []

    for _ in range(self.n_iter):
        errors = 0
        for xi, target in zip(x, y):
            update = self.eta * (target - self.predict(xi))
            self.w_[1:] += update * xi
            self.w_[0] += update # Aktualizacja biasu
            errors += int(update != 0.0)
        self.errors_.append(errors)
    return self
```

- Wagi początkowe są zerowane, w tym bias (self.w_[0]).
- Dla każdej epoki wykonywana jest pętla po wszystkich próbkach danych.

- Różnica między oczekiwaną etykietą (target) a przewidywaną wartością (`self.predict(xi)`) służy do wyliczenia update:

$$\text{update} = \eta \cdot (y_{\text{target}} - y_{\text{predicted}})$$

$$\text{update} = \eta \cdot (y_{\text{target}} - y_{\text{predicted}})$$
- Wagi są aktualizowane proporcjonalnie do tego, jak bardzo wynik odbiega od oczekiwanego:
 - $\text{self.w}_{[1:]} += \text{update} \cdot \text{xi}$
 - $\text{self.w}_{[0]} += \text{update}$ (bias).
- Jeśli `update != 0`, oznacza to, że perceptron popełnił błąd, więc errors zostaje zwiększone.
- Po każdej epoce liczba błędów jest zapisywana

Metoda `net_input`

```
def net_input(self, x):
    """Oblicza całkowite pobudzenie sieci"""
    return np.dot(x, self.w_[1:]) + self.w_[0]
```

Oblicza wartość liniowej funkcji aktywacji jako sumę iloczynów wag i cech plus bias:

$\text{net_input}(x) = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + w_0$

Metoda `predict`

```
def predict(self, x):
    """Zwraca etykietę klas po obliczeniu funkcji skoku jednostkowego"""
    return np.where(self.net_input(x) >= 0.0, 1, -1)
```

Na podstawie wyniku `net_input` zwraca:

1, jeśli wynik ≥ 0 .

-1, jeśli wynik < 0 .

Jest to implementacja funkcji skoku jednostkowego (ang. unit step function).

Przetwarzanie danych Iris

```
x, y = get_iris_binary_data()
```

Wczytanie i przygotowanie danych do klasyfikacji binarnej (tylko dla Iris-setosa i Iris-virginica).

Wizualizacja danych

```
colors = ["red" if label == -1 else "blue" for label in y]
plt.scatter(x[:, 0], x[:, 1], c=colors, marker="o")
plt.xlabel("Długość działki")
```

```
plt.ylabel("Długość płatka")
plt.title("Zbiór danych Iris")
plt.show()
```

Punkty na wykresie:

Czerwone: Iris-setosa (etykieta -1).

Niebieskie: Iris-virginica (etykieta 1).

Uczenie perceptronu dla różnych eta

```
eta_values = [0.1, 0.01, 0.001]
results = {}

for eta in eta_values:
    perceptron = Perceptron(eta=eta, n_iter=10)
    perceptron.fit(x, y)
    results[eta] = perceptron.errors_
```

Dla każdego współczynnika uczenia (eta) tworzony jest nowy perceptron.

Perceptron jest trenowany na danych, a liczba błędów w każdej epoce zapisywana w results.

Wizualizacja liczby błędnych klasyfikacji

```
for eta, errors in results.items():
    plt.plot(range(1, len(errors) + 1), errors, label=f"eta={eta}")

plt.xlabel("Epoki")
plt.ylabel("Liczba błędów")
plt.legend()
plt.title("Liczba błędnych klasyfikacji w kolejnych epokach")
plt.show()
```

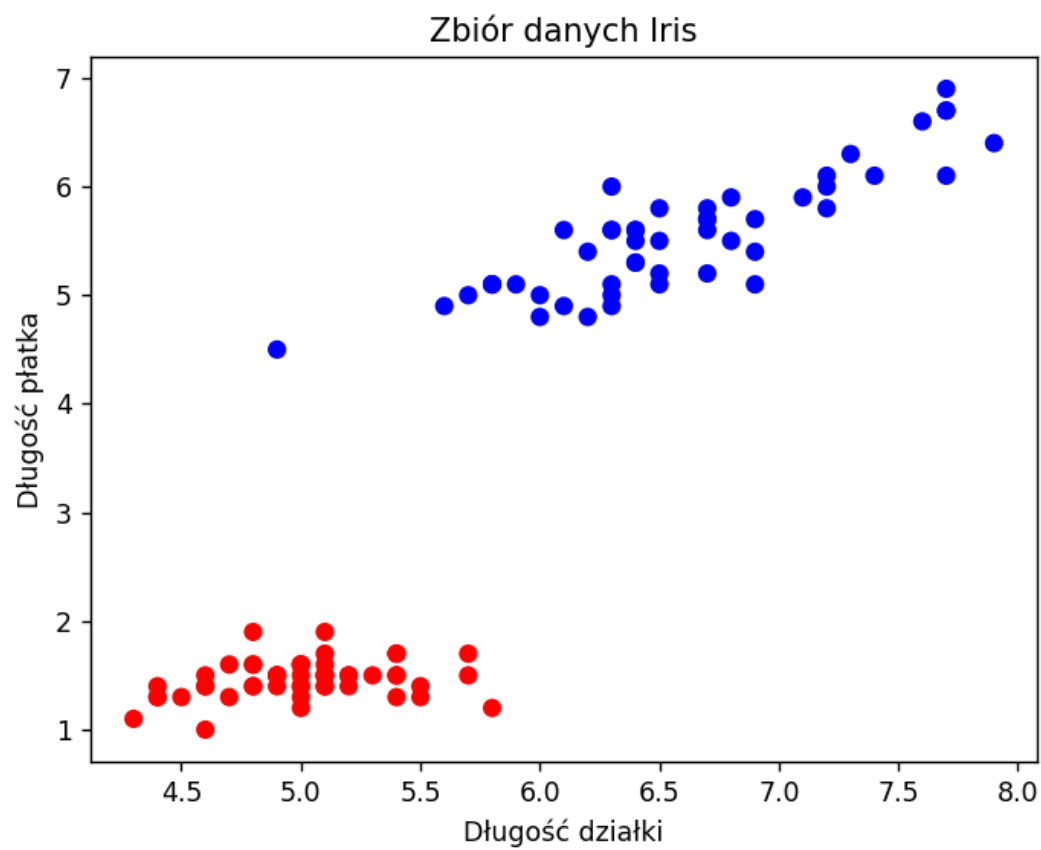
Tworzony jest wykres pokazujący, jak zmieniała się liczba błędnych klasyfikacji w zależności od epok.

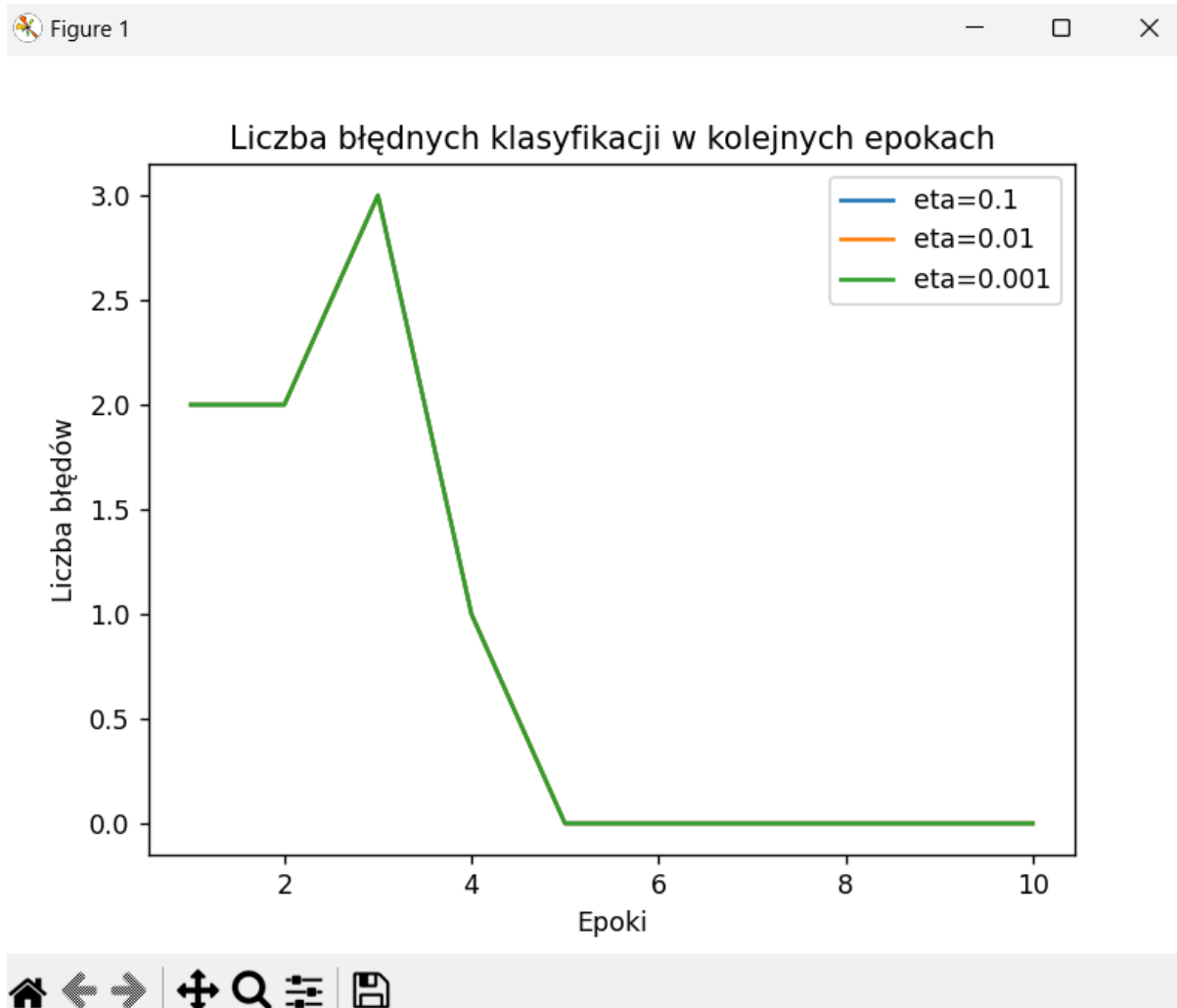
Linie reprezentują różne wartości eta:

Wyższe eta powoduje szybsze uczenie, ale większe oscylacje.

Niższe eta skutkuje wolniejszym uczeniem.

Wyniki działania Perceptronu





Algorytm implementuje perceptron jako model klasyfikacji binarnej, pokazuje proces uczenia przez aktualizację wag, wykorzystuje dane zbioru Iris do demonstracji działania perceptronu oraz ilustruje wpływ współczynnika uczenia na szybkość i skuteczność uczenia.

Jak można zobaczyć na grafikach, po 5 epochach, ilość błędów się równia 0 – oznacza to, że taka ilość generacji była wystarczająca dla t =wytrenowania perceptronu na bezbłędne działanie.

Adaline

adalineGD.py

Importowanie bibliotek i funkcji

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

from load_data import get_iris_binary_data
```

numpy: Obsługuje obliczenia macierzowe i wektoryzowane.

matplotlib.pyplot: Umożliwia wizualizację danych (np. kosztu, granic decyzyjnych).

ListedColormap: Stosowany do kolorowania regionów decyzyjnych.

get_iris_binary_data: Funkcja do ładowania przetworzonych danych zbioru Iris w wersji binarnej.

Klasa AdalineGD

```
class AdalineGD:
```

Konstruktor klasy (`__init__`)

```
def __init__(self, eta=0.01, n_iter=50):
    self.eta = eta
    self.n_iter = n_iter
    self.w_ = None
    self.cost_ = []
```

eta: Tempo uczenia się (mały krok dla precyzyjnej konwergencji).

n_iter: Liczba epok (ilość iteracji algorytmu uczenia).

w_: Wektor wag, inicjalizowany później w metodzie fit.

cost_: Lista przechowująca wartości funkcji kosztu w każdej epoce.

Metoda fit

```
def fit(self, x, y):
    """Trenowanie za pomocą danych uczących."""
    self.w_ = np.zeros(1 + x.shape[1])
    self.cost_ = []

    for _ in range(self.n_iter):
        net_input = self.net_input(x)
        output = self.activation(net_input)
        errors = y - output
        self.w_[1:] += self.eta * x.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

self.w_: Zawiera wagi modelu. Pierwszy element odpowiada biasowi.

self.cost_: Historia kosztu dla analizy zbieżności.

net_input: Oblicza pobudzenie sieci.

activation: W przypadku Adaline funkcja aktywacji to tożsamość, więc wyjście równa się pobudzeniu ($\text{output} = \text{net_input}$).

Błąd (errors): Różnica między etykietami prawdziwymi a przewidywanymi.

Aktualizacja wag i biasu:

Gradient kosztu względem każdej wagi jest równy iloczynowi tempa uczenia i błędu.

Wagi i bias aktualizowane na podstawie błędu w celu minimalizacji kosztu.

Koszt (cost): Suma kwadratów błędów podzielona przez 2.

Metoda *net_input*

```
def net_input(self, x):  
    """Oblicza całkowite pobudzenie."""  
    return np.dot(x, self.w_[1:]) + self.w_[0]
```

Oblicza liniową kombinację wag i cech wejściowych oraz dodaje wartość biasu.

Metoda *activation*

```
def activation(self, x):  
    """Funkcja aktywacji."""  
    return x
```

W Adaline funkcja aktywacji to **funkcja liniowa** (przepuszcza wartość bez zmian).

Metoda *predict*

```
def predict(self, x):  
    """Zwraca etykietę klas po wykonaniu skoku jednostkowego."""  
    return np.where(self.net_input(x) >= 0.0, 1, -1)
```

Dokonyuje prognozy klasy na podstawie pobudzenia sieci (*net_input*).

Zastosowanie funkcji progowej (skok jednostkowy):

- Wynik $\geq 0 \rightarrow$ klasa 1.
- Wynik $< 0 \rightarrow$ klasa -1.

Wizualizacja granic decyzyjnych

```
def plot_decision_regions(x, y, classifier, resolution=0.02):  
    markers = ("o", "x")  
    colors = ("red", "blue")  
    cmap = ListedColormap(colors[: len(np.unique(y))])  
    x1_min, x1_max = x[:, 0].min() - 1, x[:, 0].max() + 1  
    x2_min, x2_max = x[:, 1].min() - 1, x[:, 1].max() + 1  
    xx1, xx2 = np.meshgrid(  
        np.arange(x1_min, x1_max, resolution), np.arange(x2_min, x2_max,  
resolution)  
    )  
    z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)  
    z = z.reshape(xx1.shape)  
    plt.contourf(xx1, xx2, z, alpha=0.3, cmap=cmap)  
    plt.xlim(xx1.min(), xx1.max())  
    plt.ylim(xx2.min(), xx2.max())  
    for idx, cl in enumerate(np.unique(y)):  
        plt.scatter(  
            x=x[y == cl, 0],  
            y=x[y == cl, 1],  
            alpha=0.8,  
            c=colors[idx],  
            marker=markers[idx],  
            label=f"Klasa {cl}",
```

```
)
```

Tworzy siatkę punktów (np. meshgrid) w przestrzeni cech.
Przewiduje klasy dla każdego punktu siatki (classifier.predict).
Rysuje granice decyzyjne na mapie kolorów (contourf).
Klasy są wizualizowane jako różne kolory i markery.

Eksperymenty i wizualizacje

Ładowanie danych

```
x, y = get_iris_binary_data()
```

Dane zbioru Iris:

- **x**: Macierz cech (np. długość działki i długość płatków).
- **y**: Wektory etykiet (-1 i 1).

Trenowanie modelu dla różnych eta

```
eta_values = [0.0001, 0.1]
results = {}

for eta in eta_values:
    adaline = AdalineGD(eta=eta, n_iter=10)
    adaline.fit(x, y)
    results[eta] = adaline.cost_
```

Eksperymenty z różnymi współczynnikami uczenia:

eta=0.0001: Bardzo mały krok, wolna konwergencja.

eta=0.1: Większy krok, szybsza konwergencja.

Koszt w każdej epoce jest zapisany w results.

Wizualizacja kosztu

```
for eta, cost in results.items():
    plt.plot(range(1, len(cost) + 1), cost, label=f"eta={eta}")
```

Dla każdego eta rysowany jest wykres kosztu w zależności od epok.

Wizualizacja granic decyzyjnych

```
for eta in eta_values:
    adaline = AdalineGD(eta=eta, n_iter=10)
    adaline.fit(x, y)
    plot_decision_regions(x, y, classifier=adaline)
    plt.title(f"Granice decyzyjne dla eta={eta}")
    plt.xlabel("Długość działki")
```

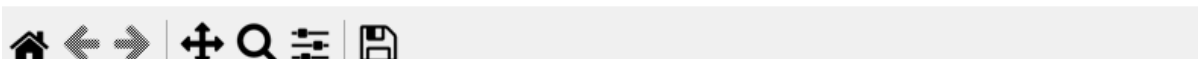
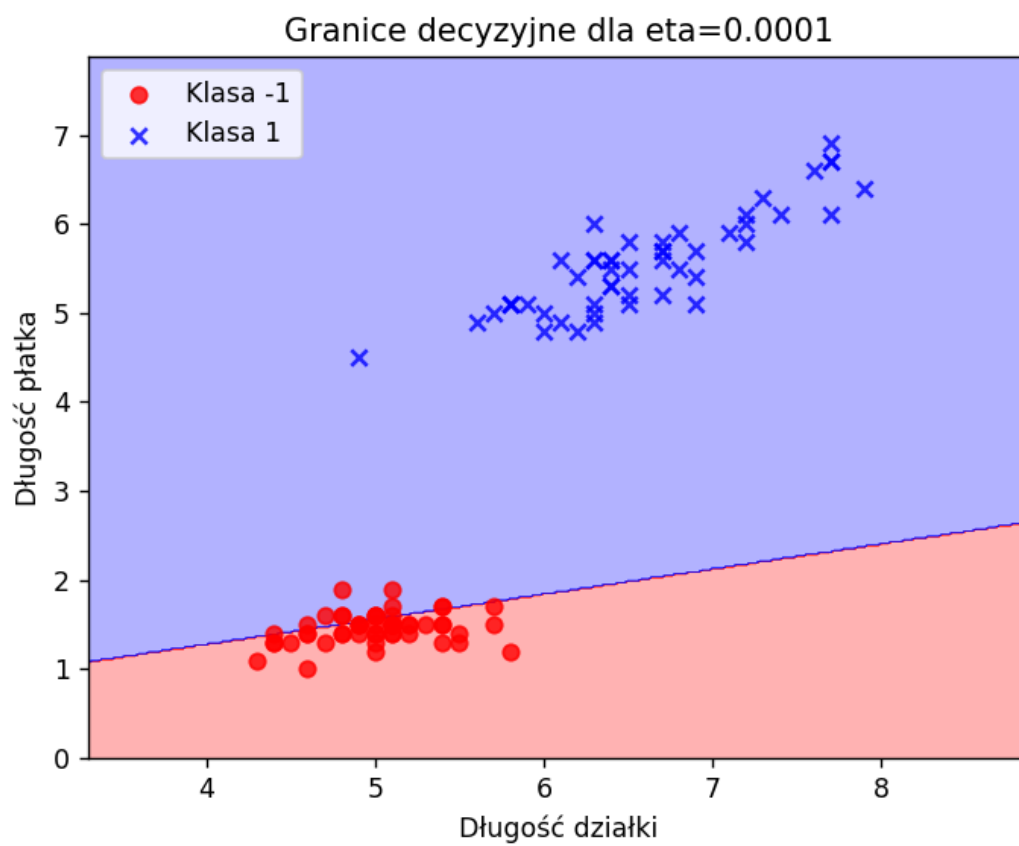
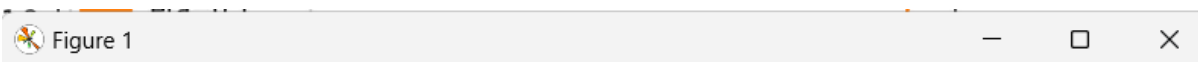
```
plt.ylabel("Długość płatka")  
plt.legend()  
plt.show()
```

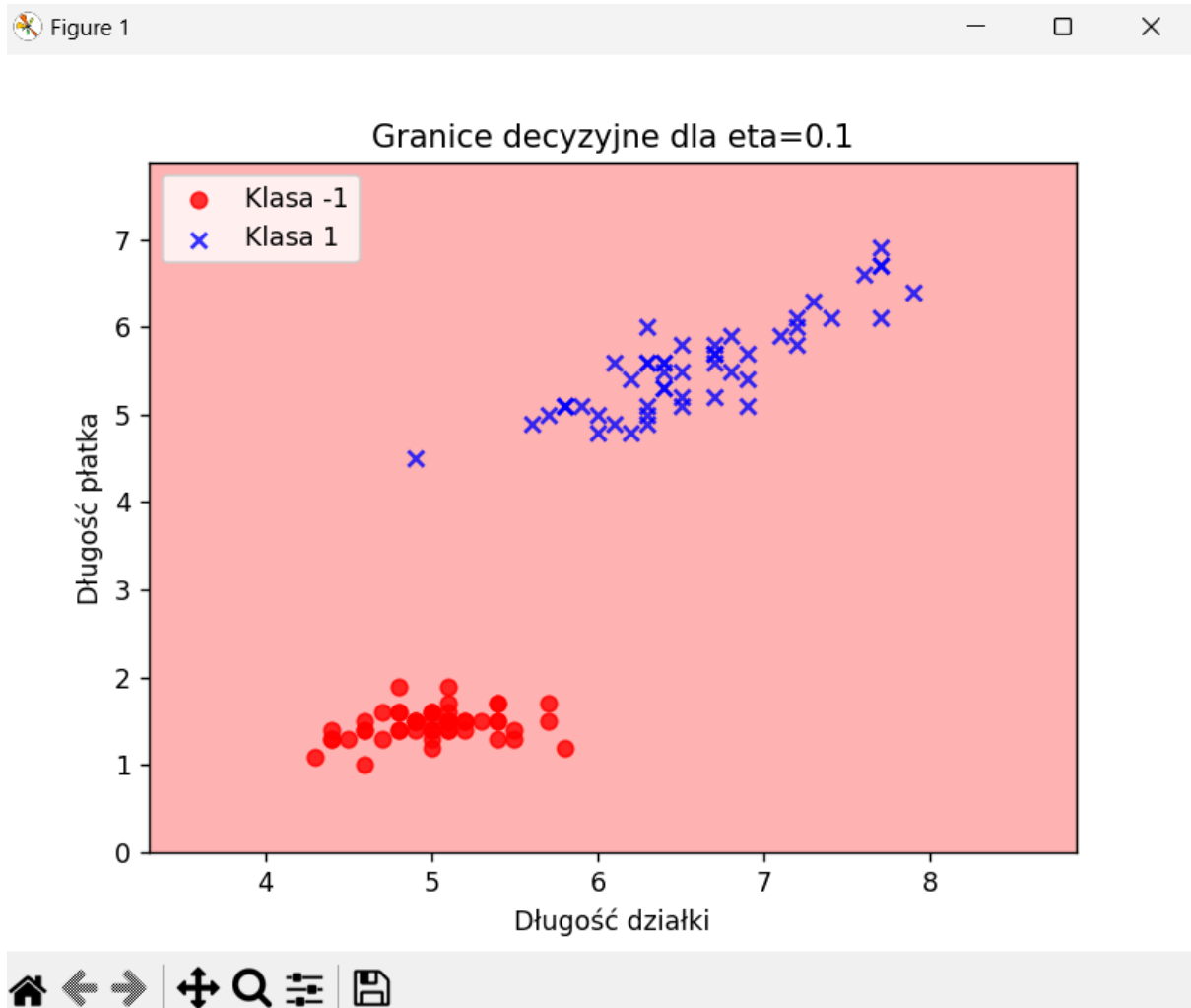
Granice decyzyjne są rysowane dla każdego tempa uczenia (η).
Kolory i markery reprezentują różne klasy.

Wyniki działania Adaline

Figure 1







Klasyfikator Adaline stosuje **gradientowy spadek** do minimalizacji funkcji kosztu. Proces uczenia obejmuje aktualizację wag na podstawie błędów w prognozach. Wykorzystanie funkcji liniowej pozwala na modelowanie problemów liniowo separowalnych. Wizualizacje kosztu i granic decyzyjnych pomagają analizować efektywność modelu dla różnych parametrów (np. eta). Przy eta ≥ 0.1 koszt będzie się zwiększał od 9 epochy.

AdalineSGD.py

Importy – wiadomo po co.

Konstruktor klasy *AdalineSGD*

```
class AdalineSGD:
    def __init__(self, eta=0.01, n_iter=10, shuffle=True, random_state=None):
        self.eta = eta
        self.n_iter = n_iter
        self.shuffle = shuffle
        self.random_state = random_state
        self.w_initialized = False
        if random_state:
```

```
np.random.seed(random_state)
```

Parametry konstruktora:

eta=0.01 – Ustala szybkość uczenia, tj. wielkość kroku, jaki algorytm wykonuje podczas aktualizacji wag.

n_iter=10 – Liczba epok, czyli pełnych iteracji przez dane treningowe.

shuffle=True – Jeśli ustawione na True, dane są losowane przed każdą epoką, co pomaga uniknąć zapamiętywania sekwencji danych.

random_state=None – Opcjonalnie ustawia ziarno generatora liczb pseudolosowych dla powtarzalności wyników.

Inicjalizacja zmiennych:

self.w_initialized – Flaga wskazująca, czy wagi zostały zainicjalizowane.

np.random.seed(random_state) – Ustawia generator liczb pseudolosowych dla kontrolowanego losowania.

Metoda fit (uczenie modelu)

```
def fit(self, X, y):
    self._initialize_weights(X.shape[1])
    self.cost_ = []
    for _ in range(self.n_iter):
        if self.shuffle:
            X, y = self._shuffle(X, y)
        cost = []
        for xi, target in zip(X, y):
            cost.append(self._update_weights(xi, target))
        avg_cost = sum(cost) / len(y)
        self.cost_.append(avg_cost)
    return self
```

Działanie:

Wywoływana jest metoda `_initialize_weights` w celu inicjalizacji wag na 0 (plus dodatkowy element `w_0` dla biasu).

Tworzona jest lista `self.cost_`, która przechowuje średni koszt w każdej epoce.

Algorytm przechodzi przez liczbę epok zdefiniowaną w `n_iter`.

Jeśli `shuffle=True`, dane są losowane w każdej epoce za pomocą metody `_shuffle`.

W każdej epoce iteruje po parach `(xi, target)` z danych:

xi to wektor cech pojedynczej próbki.

target to oczekiwana etykieta.

Wywoływana jest metoda `_update_weights`, która aktualizuje wagi i zwraca koszt dla próbki.

Średni koszt (`avg_cost`) jest obliczany i zapisywany w `self.cost_`.

Zwracana wartość:

Zwraca sam obiekt `AdalineSGD`, umożliwiając zagnieżdżone wywołania.

Metoda `_update_weights` (aktualizacja wag)

```
def _update_weights(self, xi, target):
    output = self.net_input(xi)
    error = target - output
    self.w_[1:] += self.eta * xi * error
    self.w_[0] += self.eta * error
    cost = 0.5 * error**2
    return cost
```

Działanie:

1. `net_input(xi)` oblicza liniowe wejście na podstawie obecnych wag (`self.w_`) i danych wejściowych (`xi`).
2. Obliczany jest błąd jako różnica między rzeczywistym celem (`target`) a wyjściem (`output`).
3. Wagi są aktualizowane:
 - a. `self.w_[1:]` – Wagi przypisane do cech są modyfikowane proporcjonalnie do błędu, wektora `xi` i szybkości uczenia `eta`.
 - b. `self.w_[0]` – Bias również jest modyfikowany proporcjonalnie do błędu.
4. Koszt dla próbki jest obliczany jako połowa kwadratu błędu.
5. Zwracany jest koszt dla danej próbki.

Metoda `_shuffle` (losowanie danych)

```
def _shuffle(self, X, y):
    r = np.random.permutation(len(y))
    return X[r], y[r]
```

Tworzy losową permutację indeksów za pomocą `np.random.permutation`.

Zwraca dane `X` i etykiety `y` w losowej kolejności.

Metoda `predict` (przewidywanie klasy)

```
def predict(self, X):
    return np.where(self.activation(X) >= 0.0, 1, -1)
```

Oblicza aktywację dla danych wejściowych `X` za pomocą metody `activation`.

Decyduje o etykiecie:

- 1, jeśli wartość aktywacji ≥ 0 .
- -1 w przeciwnym razie.

Pozostałe sekcji z komentarzami niżej:

```
# Funkcja do wizualizacji granic decyzyjnych
def plot_decision_regions(X, y, classifier, resolution=0.02):
    markers = ('o', 'x')
    colors = ('red', 'blue')
    cmap = ListedColormap(colors[:len(np.unique(y))])
```

```

# Rysowanie powierzchni decyzyjnej
x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                        np.arange(x2_min, x2_max, resolution))
Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
Z = Z.reshape(xx1.shape)
plt.contourf(xx1, xx2, Z, alpha=0.3, cmap=cmap)
plt.xlim(xx1.min(), xx1.max())
plt.ylim(xx2.min(), xx2.max())

# Rysowanie punktów
for idx, cl in enumerate(np.unique(y)):
    plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1],
                alpha=0.8, c=colors[idx],
                marker=markers[idx], label=f"Klasa {cl}")

# Wczytanie danych Iris za pomocą funkcji z pliku load_data.py
X, y = get_iris_binary_data()

# Wytrenowanie modelu AdalineSGD
adaline = AdalineSGD(eta=0.01, n_iter=15, random_state=1)
adaline.fit(X, y)

# Wizualizacja błędu w zależności od liczby iteracji
plt.plot(range(1, len(adaline.cost_) + 1), adaline.cost_, marker='o')
plt.xlabel('Epoki')
plt.ylabel('Średni koszt')
plt.title('Zmiana kosztu podczas trenowania')
plt.show()

# Wizualizacja granic decyzyjnych
plot_decision_regions(X, y, classifier=adaline)
plt.title('Granice decyzyjne - AdalineSGD')
plt.xlabel('Długość działki')
plt.ylabel('Długość płatka')
plt.legend(loc='upper left')
plt.show()

```

Podsumowanie wyników działania adalineSGD.py

Wykres kosztu:

- Powinien pokazywać spadek kosztu w miarę wzrostu liczby epok.
- Stabilizacja kosztu sugeruje, że model zbiega się do optymalnego rozwiązania.

Granice decyzyjne:

- Pokazuje podział przestrzeni cech na dwie klasy (1 i -1).
- Granica decyzyjna powinna być linią prostą, ponieważ Adaline to model liniowy.
- Punktacja danych powinna być zgrupowana w dwóch obszarach, co wskazuje na dobrą separację klas.